

Building the Environment, Not the Model

Practical lessons from scaling gradient-free genetic algorithms: decomposition, engineered environments, and honest failure — observed across a multi-month specialist-genome pipeline.

FIELD NOTES • GENREG PROJECT • 2026-07-07 • NOT PEER REVIEWED — AN EMPIRICAL LOG, NOT A CONTROLLED STUDY

ABSTRACT

Gradient-free genetic algorithms (GAs) do not scale by searching a bigger space harder; they scale by searching a smaller, better-shaped space repeatedly. Across roughly thirty trained genomes in a single pipeline — some shipped, most cut — the pattern that reliably worked was: build the representation space from ordinary corpus/domain statistics (fixed, not evolved), then hand a tiny evolved population one narrow, falsifiable question about that space, scored against hard, structured negatives rather than random ones. Complex end-to-end behavior emerged from composing dozens of these tiny specialists (14 to 3,100 parameters each) rather than growing any one of them. We report what worked (a handful of clean validations, one successful problem reframing that fixed a specific failure mode, and a working scheme for distributing independent GA training runs across heterogeneous machines) and, in more detail, what did not — because the failures were more informative than the successes about where this approach's boundaries are. Roughly two-thirds of the genomes attempted were cut. The paper's claim is narrow and empirical, not a general theory of evolutionary computation: within this pipeline, representation engineering plus honest battery-testing beat trying to make any single genome smarter.

1 The Core Wager

A genetic algorithm searches by mutating and selecting among a population of candidate parameter sets, scored by a fitness function, with no gradient anywhere. That is a powerful idea and a narrow one: population-based search is hopeless against a space with millions of free parameters and a diffuse reward signal, which is exactly the regime modern gradient-trained models occupy. It is a different story against a space with a few hundred parameters and a sharp, well-posed reward. The practical question this project kept returning to was not “how do we make the GA smarter” but “how do we make the space small and sharp enough for the GA we already have.”

The answer that kept working, across a pipeline that eventually spanned roughly thirty trained components, was to stop asking evolution to discover structure and start handing it structure that was

already there. Word classes were induced with k-means over co-occurrence statistics, not evolved. Distributional word features were built with SVD over a corpus, not evolved. Coordination context, function-word membership, morphological suffixes — all fixed, all computed once from data, all handed to the GA as a read-only environment. What evolution was asked to do, every single time, was find one small bilinear or linear relationship inside that pre-built space: does this pair of vectors satisfy this one relation or not. That division of labor — statistics builds the world, evolution finds one path through it — is the one idea this paper generalizes.

The wager, stated plainly

Don't evolve the embedding space. Build it from domain statistics. Evolution then learns one tiny relationship inside a space someone else already shaped. The features are the environment; the population is the organism navigating it — and an organism only needs to be smart enough for the environment it is actually in.

This is a report from inside one such pipeline: a language-generation system built as a stack of narrow, gradient-free “genomes” rather than one trained network. The domain is incidental to the argument. What follows is deliberately not a paper about language modeling — it is a paper about what happened, mechanically, when dozens of tiny evolutionary searches were run, measured, and either kept or discarded, and what that process implies about scaling GA-based systems in general.

2 The Specialist-Genome Pattern

2.1 One genome, one falsifiable question

Every component in the pipeline followed the same shape. A genome is a small evolved object — usually a bilinear matrix ($\text{score} = f(a) M f(b)$), sometimes a linear vector, occasionally a tiny two-layer scorer over an embedded window — with a population of 100–200 candidates, evolved by tournament selection ($k=4$), 10% elitism, an 8% starvation cull, and self-adaptive per-candidate mutation step size clipped to $[0.005, 0.4]$. Sizes ranged from 14 parameters (a unary sentence-opener classifier) to about 3,100 (a next-grammatical-class predictor). None used a gradient at any point. What made a genome well-posed, in the cases that worked, was that its entire reason to exist could be stated as one sentence someone could falsify: does the subject agree with the verb, is X a type of Y, does this word start a sentence. A genome without a single sharp reason to exist reliably underperformed one with a narrow, correctly-scoped question — restating a broader goal (“be more grammatical”) never worked as a fitness function; narrowing it until it was answerable from the features on hand always did.

2.2 The environment is engineered, not learned

Two feature spaces recurred throughout the project, both built once from raw corpus statistics and then frozen: a small in-pipeline distributional space (24–128 dimensions, SVD over local co-occurrence within a few thousand words) used by the generation-time genomes, and a larger standalone space (128 dimensions, SVD over a 51-million-word corpus, 30,000 vocabulary) built specifically to give later genomes enough lexical coverage for relations a small corpus cannot expose — hypernymy needs to have

seen “diabetes” and “disease” together; a corpus of 19th-century novels simply never will. Building a bigger, cleaner environment turned out to be the highest-leverage move available whenever a genome was starved for signal — more directly effective, run for run, than tuning the GA's own hyperparameters.

2.3 Fitness via hard-negative discrimination

The fitness function in almost every surviving genome was the same shape: score real pairs/sequences from the corpus against hard negatives — corrupted examples drawn from a distribution close enough to the real one that only the target relation can tell them apart — rather than uniformly random negatives, which are almost always trivially separable on some irrelevant axis. This distinction was not a minor tuning detail; it was the difference between a genome that looked correct on its own metric and one that was actually correct.

Failure that taught the lesson: agreement, first attempt

The subject-verb/modal-aux agreement genome was first trained with uniform-random word negatives and 17 coarse features that could not distinguish *be* from *is*. It scored an inflated 0.824 global discriminator accuracy — and passed only 6 of 12 held-out minimal pairs (“he could be” vs “he could is”). Rebuilt with hard negatives (same preceding word, candidate resampled from the real next-word distribution) and 22 finiteness-aware features, global accuracy dropped to 0.587 — correctly near-chance, since most adjacent word pairs carry no agreement constraint at all — while minimal-pair accuracy went to 12 of 12. The higher global number was the worse genome.

2.4 The battery: build many, ship few, log the losers

Genomes were built in batches against a roadmap, trained, gated first on whether they beat chance on held-out validation, then — the step that mattered — measured for an actual effect on the thing they were meant to fix, with guardrails against breaking anything else. A genome that passed its own validation metric but moved nothing when wired into generation was cut. A genome that scored weakly on validation but measurably improved a real generation metric with no downside was shipped. This asymmetry — the validation metric is necessary but not sufficient, in both directions — is discussed at length in Section 4, because it was the single most consistent surprise of the project.

3 Results — What Worked

3.1 Structural decomposition: measured, not assumed, gains

The first battery targeted purely structural (grammar/position/rhythm) constraints. Thirteen of these genomes shipped into the live generation pipeline; total footprint across all of them was about 9,500 evolved parameters plus a small phrase lookup table — on the order of a few hundred kilobytes deployed, not gigabytes. Representative results:

GENOME	PARAMS	MEASURED EFFECT
Vocabulary	~2,885	Valid-word rate 19% 52%
Order (next-class)	~3,108	Perplexity 10.91, ~75% of the estimated ceiling
Selection (bigram fit)	~577	Adjacency-fit +15 points
Bidirectional selection	~1,150	Adjacency-fit 0.777, distinct-word ratio 0.231
Alternation (content/function)	~200	Illegal function-function pairs 57.5% 37.8%
Agreement	~480	12/12 held-out minimal pairs (see §2.3)
Semantic co-occurrence	~580	Real content-word adjacency 33% 40%, no fluency cost
No-repeat	~10	Content-word repetition 2.2% 0%
Sentence opener	~14	Bad sentence-openings 17% 8%
Sentence closer	~14	Bad sentence-endings 65% 33%, rate-preserving

Table 1. A representative slice of the shipped structural battery. Full roadmap and the cut/deferred majority in the project's internal genome log.

Two mechanical findings from this battery generalized well beyond their specific genomes. First, redundancy is the main killer, not incapacity: several well-trained genomes (clause-template validity, verb-argument binding, pronoun-antecedent binding) were cut not because they failed to learn their target relation, but because a higher-context genome already sitting in the pipeline (a 4-class-lookback next-class predictor) already implied the same constraint at the level the new genome operated on. A class-level windowed validity check is always subsumed by a sufficiently wide next-class context model; it can only add value where it sees something the wider model structurally cannot (word identity, not class; persistent state, not a fixed window). Second, a genome that looked structurally similar to a working one could still fail for an orthogonal reason: a determiner-noun binding genome learned its target relation cleanly (0.776 discriminator accuracy, eliminated orphaned determiners) but was still cut, because its features were too coarse to pick the correct noun — it fixed one defect while quietly introducing off-distribution bigrams elsewhere, a net loss the validation metric alone never revealed.

3.2 A clean semantic-relation validation: hypernymy

A second, independent battery asked whether the same recipe — build a distributional space, evolve one relation inside it — extends to lexical-semantic relations (is-a, part-of, synonym, antonym) rather than purely structural ones. Positive pairs were mined from a 51-million-word corpus with simple surface patterns (Hearst patterns for hypernymy: “X such as Y,” “X and other Y”), inherently noisy, then a directional bilinear genome was trained to prefer $isa(x,y)$ over $isa(y,x)$ against both a reversed pair and a random pair. The result held up on hand-picked probes never seen in mining or training — the only test that matters, discussed in §4.1:

PROBE PAIR (X → Y)	$isa(x,y)$	$isa(y,x)$	CORRECT?
dog animal	-0.24	-2.74	yes

PROBE PAIR (X → Y)	isa(x,y)	isa(y,x)	CORRECT?
france country	+0.99	-3.33	yes
diabetes disease	+0.05	-5.14	yes
hammer tool	+0.39	-3.35	yes
guitar instrument	-0.40	-1.42	yes

Table 2. Hypernym genome, 5 of 10 held-out probes shown (10/10 passed). Directional held-out accuracy 0.86. Note the absolute scores are often small or negative — what the genome reliably learned was the ordering, not a calibrated magnitude.

This is the paper's cleanest positive result: a relation that is genuinely hard to state as a simple corpus statistic (hypernymy is directional; co-occurrence alone is symmetric) was recovered by a ~16,000-parameter bilinear head trained on noisy, automatically-mined pairs, and it generalized to hand-picked examples with no overlap with the training mining patterns. The abstraction did not need clean labels to work; it needed a feature space rich enough to make the relation linearly separable once oriented correctly, and 1,200 generations of selection to find that orientation.

3.3 Reframing a failure into a working genome

Training separate synonym and antonym detectors failed a decisive test each: the synonym genome ranked an intentionally unrelated control pair above real synonym pairs (it had learned “related,” not “same-meaning”); the antonym genome scored its own flagship example, hot/cold, in the wrong direction. Both failures trace to the same root cause — synonyms and antonyms are distributionally almost identical (both pairs occur in similar contexts), so a detector that only sees “this pair is related” cannot separate them.

The fix was not a bigger genome or more training — it was a different question. Rather than training two genomes to each answer “is this pair X,” one genome was trained on the union of both mined sets to answer a strictly narrower question: given that a pair is already known to be related, is it same-meaning or opposite-meaning? No unrelated pairs ever entered training. This removed the confound the two separate genomes were quietly fighting.

Reframing result

The flagship failure case, hot/cold, flipped from -0.29 (wrong direction) to +0.86 (correct) under the unified reframing. Held-out same/opposite accuracy reached 0.77, and 11 of 14 hand-picked probes were correct — including every clear-cut antonym pair tested except one. A specific, diagnosable residual failure remained (size-adjective synonym pairs like big/large still misclassified as antonym-leaning, plausibly because they also co-occur in real coordination contexts like “big and small businesses”, contaminating the exact signal the genome relies on) — reported honestly as a partial result, not shipped as a clean win.

The general lesson is broader than this one pair of genomes: when two target classes are confusable along one axis, do not ask a single discriminator to solve both the confound and the target distinction in the same fitness function. Where possible, factor the confound out as a precondition (here: given

relatedness) and train narrowly on the distinction that remains. This is decomposition applied recursively — the same instinct that motivated specialist genomes in the first place, applied one level deeper when a single specialist turns out to still be answering two questions at once.

3.4 Scaling the search itself: distributed, deterministic dispatch

Because every genome in this pipeline is CPU-only NumPy — population sizes of 100–200, feature dimensions of a few dozen to a few hundred — a single training run costs minutes to tens of minutes, and different genomes are fully independent of each other. That makes the workload embarrassingly parallel across machines in a way gradient-trained systems usually are not (where parallelism has to be engineered into a single model's training loop). A lightweight job-dispatch layer was built on top of infrastructure already used for signed code deployment (Ed25519-signed job submission, a per-machine FIFO queue so a single machine's runs don't contend with each other, cancel/retry/pause controls), and three independent genome retrains were split two-machines-to-one without any special-casing: the same fixed random seed on each machine reproduced bit-for-bit the same validated result already measured on a single machine, because nothing about the training loop is machine-dependent. Horizontal scaling of a GA-based system, in other words, is closer to scaling a batch job queue than to scaling a single distributed training run — a simpler problem, with a correspondingly simpler solution.

4 Results — What Did Not Work, and Why

Roughly two out of every three genomes attempted in this project were cut. That ratio is reported deliberately: a pipeline that only reports its survivors looks like a much cleaner process than genetic search actually is. What follows are the four recurring failure modes, because each generalizes to a different part of a GA-based system's design.

4.1 The validation metric is not the verdict

The single most consistent surprise of the project was that a genome's own held-out/validation accuracy predicted almost nothing about whether it was a good genome, in either direction.

GENOME	VALIDATION METRIC	REAL-WORLD OUTCOME
Semantic co-occurrence	0.557 (weak)	SHIPPED — real generation-time gain, no cost
Determiner binding	0.776 (strong)	CUT — fixed one defect, broke fluency elsewhere
Polysemy classifier	0.88 (strong)	CUT — failed every hand-picked probe
Unified synonym/antonym	0.77 (moderate)	PARTIAL — real gain, one diagnosed residual failure

Table 3. Validation accuracy vs. outcome after the actual behavioral test. The correlation between the two columns, across this project, was weak in both directions.

The reason is structural, not a fluke of these particular genomes: validation accuracy measures fit to the mined training distribution, and mined distributions are themselves imperfect proxies for the target concept (see §4.2). A genome can fit its proxy well and miss the concept, or fit its proxy weakly —

because the proxy itself is noisy — while still capturing enough of the real signal to move the metric that actually matters. The operational consequence is that a battery process cannot gate on validation accuracy alone; it has to measure the genome's effect on the actual downstream behavior it was built to change, with guardrails to catch collateral damage the target metric would not show.

4.2 Label quality dominates architecture

Every genome in this project is only as good as the positive examples it was mined to recognize, since none of the relations attempted have ground-truth labels available at corpus scale — mining a proxy signal from surface patterns is the whole game. Three clean failures trace entirely to bad proxies rather than a learnability problem:

Sentiment (monolithic positive/negative)

Nearest-neighbor propagation from a small positive/negative seed list drifted onto generic high-frequency words instead of true polarity. The trained genome inverted its own probes: “war” scored +7.85, “joy” scored -7.38. Not a marginal miss — the mining recipe itself produced the wrong labels.

Polysemy (does this word have multiple senses)

A nearest-neighbor-similarity-spread proxy was used as a stand-in for “how many senses does this word have,” on the assumption that a polysemous word's neighbors scatter across its senses. On this corpus, classic polysemous words (bank, spring, bat, star) are used so overwhelmingly in one dominant sense that their neighbor cluster stays tight — the proxy read them as monosemous. Validation accuracy was 0.88; every hand-picked probe failed.

Register (formal narration vs. informal dialogue)

Quote-span mining (words inside quotation marks vs. outside) was meant to separate dialogue register from narration register. It instead separated “words common in narration about feelings” from everything else — a different, unintended split. Weak validation accuracy (0.61) and incoherent probes followed directly.

In every one of these cases the training machinery — the bilinear/unary genome, the GA loop, the hard-negative discipline — was unchanged from the genomes that worked. The failure lived entirely upstream, in how the positive examples were defined. This is the sharpest practical implication for anyone scaling a similar system: budget for the labeling recipe as the primary risk, not the model.

4.3 The representation-altitude problem

One failure did not trace to bad labels or a redundant genome, and is worth treating separately because it points at a structural limit of the whole approach. An “analogy” genome (does pair A:B share the same relation as pair C:D) was trained directly on raw distributional offset vectors — the same feature space every other relation genome used — and scored at chance (0.48–0.53) through more than 600 generations with no trend in either direction.

Diagnosis: a layer-3 question asked with layer-2 machinery

“A is to B as C is to D” is not a relation living in the same space as is-a or part-of; it is a question about those relations — do pair (A,B) and pair (C,D) instantiate the same relation? That is a property of what a relation genome's output would say about each pair, not a property expressible as a fixed offset in a raw distributional space. Training directly on $\text{feat}(B) - \text{feat}(A)$ asks a question one level of abstraction higher than the representation exposes. Chance-level accuracy is the expected outcome of that mismatch — not evidence the relation is unlearnable, and not a reason to retry the same recipe with more generations or a bigger population. The corrected approach (not yet built at time of writing) scores agreement between two already-validated relation genomes' outputs — $\text{hypernym}(a_1, b_1)$ vs. $\text{hypernym}(a_2, b_2)$ — rather than raw feature offsets: a genome over genome outputs, one layer higher than everything else in this paper.

The general principle: when a genome sits at chance for a well-posed relation despite adequate mining volume and a feature space that has worked for related problems, check whether the target relation is expressible at all in the representation before concluding the search failed. A search that cannot find a signal that provably is not there will look identical, in its training curve, to a search that gave up too early — the training curve alone cannot distinguish the two, which is exactly why every genome in this project was ultimately judged on external probes rather than internal metrics.

4.4 Measurement artifacts look like results if you let them

A smaller, easily-overlooked failure mode: an early clause-validity genome appeared to score a perfect hit rate on its class-trigram metric at every tested weighting — not because the genome was correct, but because the metric itself (33 induced word classes over 8.5 million words) made nearly every class-trigram in existence appear at least once in the corpus, so “hit rate” was saturated regardless of what the genome did. Switching to a frequency-weighted log-probability metric revealed the genome was, if anything, making generation slightly worse. The lesson generalizes past this one metric: any discrete hit-rate metric over a coarse, high-cardinality space should be checked for ceiling saturation before it is trusted as a signal at all.

5 Principles for Scaling GA-Based Systems

Extracted from the pattern of successes and failures above, in the order a system designer would actually encounter them.

1. Shrink the space before you search it.

The highest-leverage lever available to a GA is not the GA's own hyperparameters (population size, mutation schedule, selection pressure) — it is how much of the problem is solved by fixed, pre-computed structure before evolution ever runs. Every genome in this project that struggled improved more from a better feature space than from more generations.

2. One genome, one falsifiable question.

A fitness function that restates a broad goal (“be grammatical,” “be meaningful”) is not a fitness function a small population can climb. A fitness function that states a single answerable question (“does the subject agree with the verb”) is.

3. Hard negatives, not random ones.

A discriminator's negatives should be as close as possible to the positives on every axis except the one being tested. Random negatives make almost any genome look artificially good by letting it succeed on an irrelevant, easy axis instead of the target relation.

4. Gate on the real effect, not the proxy metric.

Validation accuracy against a mined or synthetic label set is necessary but not sufficient in either direction. Measure the genome's actual effect on the behavior it was meant to change, with guardrails for collateral damage, before shipping it.

5. Redundancy, not incapacity, kills most candidates.

A genome that learns its target relation cleanly can still be cut if a broader-context genome already sitting in the system implies the same constraint. Decompose along axes the rest of the system structurally cannot see — exact identity, not class; persistent state, not a fixed window; a different relation, not a rescaled version of one already covered.

6. Label/mining quality is usually the actual bottleneck.

When a genome fails outright — not marginally, but invertedly or incoherently — the training machinery is rarely the cause. Check what the positive examples actually are before concluding the relation is unlearnable.

7. Match the question to the representation's altitude.

A relation that is a property of other relations (analogy, consistency, meta-relations generally) cannot be recovered from raw first-order features, no matter how long the search runs. Build it on top of already-validated lower-level genomes' outputs instead.

8. When two classes are confusable, factor out the confound before training the distinction.

If a discriminator keeps failing on a specific axis, check whether it is secretly being asked two questions at once. Narrowing the question (“given X, is it A or B” instead of “is it A, B, or neither”) can turn a failed genome into a working one without adding a single parameter.

9. Report the cut list, not just the survivors.

A battery process that only publishes what shipped will look like a cleaner, more predictable process than genetic search actually is, and will mislead the next person who tries to scale it. The failures in §4 were more informative than the successes in §3 about where this approach's boundaries sit.

10. The search is parallel even when the model isn't.

Because each genome is small, independent, and CPU-bound, a fleet of machines running one genome each scales a GA-based system further, faster, than trying to make any single training run bigger — and fixed seeds make that distribution deterministic and free to verify.

6 Where This Generalizes, and Where It Probably Doesn't

Nothing about the pattern reported here is specific to language. The recipe — build a fixed representation from domain statistics, evolve a small discriminator over hard negatives inside it, gate on real downstream effect rather than proxy accuracy, decompose along axes the rest of the system cannot already see — applies wherever a domain offers (a) cheap, plentiful unlabeled or weakly-labeled data to build a representation from, and (b) a target behavior that decomposes into many narrow, separately-verifiable constraints rather than one diffuse objective. Tabular business rules, game-state heuristics, control policies with interpretable sub-behaviors, and constraint-satisfaction problems with structured negative examples all plausibly fit that shape.

It generalizes less well wherever the target behavior genuinely does not decompose — where the interesting signal is exactly the interaction between many components rather than any one of them, or where no cheap unlabeled data exists to build a representation from in the first place. The analogy failure in §4.3 is a specific instance of a broader boundary: this approach recovers relations that are linearly (or near-linearly) separable in some representation reachable from statistics someone can compute. Relations that are only separable in the joint output space of several other not-yet-built models are reachable, but only by building the pipeline in the right order — not by making one genome bigger.

7 Limitations

This is an empirical log from one pipeline, not a controlled study, and should be read that way. Several limitations bear directly on how much weight the findings above should carry:

- Probes were engineer-selected, not held out from a benchmark. The hand-picked probe pairs used to validate relation genomes (Tables 2–3) were chosen by the same people who built the genomes, which is a weaker standard than an independently-curated benchmark, even though they were never seen during mining or training.
- Population sizes and generation counts were fixed by convention (100–200 population, 1,000–2,500 generations), not swept. Some cut genomes might have crossed a usefulness threshold with substantially more compute; the project's operating assumption — that a well-posed genome converges within this budget or the representation is the problem, not the budget — was not independently verified against a compute-scaling curve.
- “Shipped” does not mean independently peer-verified. Metrics like adjacency-fit and bad-opener-rate were defined and measured by the same project reporting the results.
- Single domain, single corpus family per experiment. All relation-genome results come from one Wikipedia-derived corpus and one novel-corpus vocabulary; generalization across corpora or languages was not tested.
- The distributed job-dispatch result (§3.4) demonstrates reproducibility across two machines on one LAN, not a scaling curve. Whether this pattern holds at fleet scale, under real network partition/failure conditions, is an open question the project has not yet exercised.

8 Conclusion

Across roughly thirty attempted genomes, the ones that survived a genuine battery test shared a shape: a narrow, falsifiable question, asked inside a representation built from real domain statistics rather than discovered by the search itself, scored against negatives hard enough that only the target relation could separate them, and judged on a real downstream effect rather than its own validation number. The ones that failed mostly failed for reasons that had nothing to do with the genetic algorithm itself — bad mining, redundant coverage, or a question posed at the wrong representational altitude — which is the most useful finding in this report, because it means the failures are fixable by changing what is fed to the search, not by making the search cleverer.

“Scaling” a system built this way looks less like scaling a single model and more like running a disciplined battery process: build the environment, ask one question at a time, measure the real effect, keep what earns its place, and — the part most reports skip — write down why the rest didn't.

Field notes compiled from a working specialist-genome pipeline. All figures in this report are measured values from that pipeline's training/evaluation logs at time of writing, reported as observed — not benchmarked against external baselines.